

Apostila do Laboratório de Linguagens de Programação

Módulo C++: Programação Orientada a Objetos

Pedro Henrique Ribeiro Dias

23 de setembro de 2026

Sumário

1	Relatório 6: Programação Orientada a Objetos em C++	2
1.1	Fundamentos Teóricos e Pilares da POO	2
1.1.1	A Mudança de Paradigma: Da Programação Estruturada à Orientação a Objetos	2
1.1.2	Os Blocos Fundamentais: Classes, Objetos, Estado e Comportamento	2
1.1.3	Os Quatro Pilares da Orientação a Objetos	3
1.2	Documentação Sintática das Estruturas de POO	5
1.2.1	Declaração de Classe e Modificadores de Acesso	5
1.2.2	Construtores e Lista de Inicialização	5
1.2.3	Herança Pública	5
1.2.4	Polimorfismo Dinâmico: <code>virtual</code> e <code>override</code>	5
1.2.5	Destrutor Virtual (<code>virtual ~Destrutor()</code>)	6
1.2.6	Ponteiros de Classe Base e Gerenciamento Dinâmico de Memória	6
1.3	Código de Referência Completo	6
1.4	Exercícios Práticos do Módulo	8
2	Relatório 7: Programação Orientada a Objetos em C#	9
2.1	Fundamentos Teóricos e Pilares da POO	9
2.2	Documentação Sintática das Estruturas de POO	9
2.2.1	Estrutura do Programa e Saída no Console	10
2.2.2	Declaração de Classe e Propriedades	10
2.2.3	Construtores e a Palavra-chave <code>this</code>	10
2.2.4	Herança e Repasse ao Construtor Base (<code>: base(...)</code>)	11
2.2.5	Polimorfismo: <code>virtual</code> e <code>override</code>	11
2.2.6	Reaproveitando o Comportamento do Pai: <code>base.Metodo()</code>	11
2.2.7	Composição e Agregação	12
2.2.8	Coleções Genéricas: <code>List<T></code> e <code>foreach</code>	12
2.3	Código de Referência Completo	12
2.4	Exercícios Práticos do Módulo	16

Capítulo 1

Relatório 6: Programação Orientada a Objetos em C++

1.1 Fundamentos Teóricos e Pilares da POO

1.1.1 A Mudança de Paradigma: Da Programação Estruturada à Orientação a Objetos

Para compreender a Programação Orientada a Objetos (POO), é fundamental entender a evolução na forma de estruturar o raciocínio computacional. Na programação estruturada ou procedural, o código é organizado como uma sequência linear ou hierárquica de procedimentos e rotinas que operam sobre dados armazenados em estruturas separadas. Nesse modelo, existe uma separação clara entre "quem executa a ação" (a função) e "quem sofre a ação" (os dados).

Conforme os sistemas de software crescem em escala e complexidade, essa separação gera desafios significativos de manutenção. Alterar a estrutura de um dado pode exigir a modificação de dezenas de funções espalhadas pelo código, aumentando o risco de efeitos colaterais indesejados.

A POO propõe uma mudança fundamental de perspectiva: em vez de tratar dados e funções como entidades separadas, ela os unifica em estruturas coesas e autônomas. O foco da modelagem deixa de ser a sequência de passos executados pelo computador e passa a ser as **entidades do domínio do problema** e as interações entre elas.

1.1.2 Os Blocos Fundamentais: Classes, Objetos, Estado e Comportamento

Antes de analisar os pilares da POO, é necessário dominar os quatro conceitos estruturais que sustentam esse paradigma:

- **Classe (O Modelo Conceitual):** Uma classe é uma especificação abstrata, funcionando como um projeto ou molde. Ela não ocupa espaço operacional como uma entidade ativa em memória; em vez disso, define a estrutura lógica, as propriedades que os elementos desse tipo possuirão e as ações que poderão executar. Classes podem abstrair conceitos do mundo real para o código, por exemplo, podemos criar uma classe *Animal*, *Pessoa*, *Carro*, *ContaBancária* e muito mais, portanto não se prendam ao modelar classes.
- **Objeto (A Entidade Concreta):** Um objeto é a concretização prática de uma classe — processo conhecido como *instanciação*. Se a classe é a planta arquitetônica de uma casa, o objeto é a casa física efetivamente construída na memória do computador durante a execução do programa. Um programa em POO consiste em um conjunto de objetos interagindo por meio do envio de mensagens. Continuando o exemplo dado no final da classe, a instância de um *Animal* poderia ser um cachorro real, por exemplo o Scooby-doo (não confundam estes conceitos, poderíamos criar uma classe *Cachorro*, uma abstração

deste animal para o código, e o objeto criado, a instância dela seria o cachorro Scooby-doo!)

- **Atributos e Estado:** Os atributos são as variáveis internas pertencentes a um objeto que armazenam suas características ou dados. O conjunto de valores armazenados nesses atributos em um determinado instante do tempo define o **estado** do objeto. O estado de um objeto é dinâmico e pode se alterar ao longo do ciclo de vida da aplicação. Por exemplo, mantendo a linha de Animais, estes podem ter espécie, nome e diversas outras características expressas em seus atributos.
- **Métodos e Comportamento:** Os métodos são as rotinas ou funções associadas a uma classe que operam sobre os seus atributos. Eles representam o **comportamento** do objeto, definindo as ações que ele pode realizar, as transformações que pode aplicar ao seu próprio estado ou as informações que pode fornecer a outros objetos. Por exemplo, mais uma vez em animais, um comportamento para um cachorro poderia ser andar(), latir(), rosnar() e tudo mais que fizer sentido para quem estiver modelando a classe.

1.1.3 Os Quatro Pilares da Orientação a Objetos

A POO se sustenta sobre quatro pilares teóricos fundamentais. A aplicação conjunta desses pilares permite a construção de softwares mais modulares, flexíveis, reutilizáveis e fáceis de manter.

1. Abstração

A abstração é o processo mental de isolar os aspectos essenciais de uma entidade do mundo real ou do domínio do problema, ignorando os detalhes irrelevantes para o contexto do sistema.

No desenvolvimento de software, não é viável nem necessário representar uma entidade em sua totalidade física ou biológica. A abstração estabelece o **fronteira do contexto**:

- Em um sistema acadêmico, a abstração de uma entidade "Pessoa" focará em atributos como matrícula, histórico de notas e curso.
- Em um sistema hospitalar, a mesma entidade "Pessoa" será abstraída focando em tipo sanguíneo, histórico médico e alergias.

A abstração correta garante que as classes contenham apenas o que é estritamente necessário para que o sistema cumpra seu propósito.

2. Encapsulamento

O encapsulamento é o mecanismo que agrupa dados (atributos) e procedimentos (métodos) em uma única unidade (o objeto) e **restringe o acesso direto** ao estado interno do objeto a partir do ambiente externo.

Os objetivos centrais do encapsulamento são:

- **Proteção de Dados:** Evitar que o estado interno do objeto seja alterado para valores inválidos ou incoerentes por agentes externos.
- **Ocultação da Implementação:** Permitir que a lógica interna de um método seja alterada pelo desenvolvedor sem impactar o restante do sistema, desde que a forma de interagir com ele continue a mesma.

O encapsulamento é operacionalizado por meio de **modificadores de acesso** (ou níveis de visibilidade):

- *Privado (Private)*: Os membros só podem ser acessados ou alterados por métodos da própria classe.
- *Público (Public)*: Os membros compõem a *interface pública* do objeto e podem ser acessados por qualquer outra parte do sistema.
- *Protegido (Protected)*: Os membros são acessíveis pela própria classe e por suas subclasses derivadas.

A comunicação segura com um objeto encapsulado ocorre mediante uma interface pública controlada, composta por métodos de validação, consulta e alteração.

3. Herança

A herança é um mecanismo de associação hierárquica que permite criar novas classes a partir de classes já existentes. Ela estabelece uma relação semântica do tipo "**é um**" ou "**é uma espécie de**".

Nessa estrutura:

- **Classe Base (Superclasse ou Classe Mãe)**: Define as características e comportamentos genéricos compartilhados por uma família de entidades.
- **Classe Derivada (Subclasse ou Classe Filha)**: Herda automaticamente a estrutura e o comportamento da classe base, podendo adicionar novos atributos/métodos específicos ou redefinir comportamentos herdados.

O principal benefício da herança é a **reutilização de código** e a organização hierárquica do conhecimento no sistema. Quando um comportamento comum a várias classes precisa ser alterado, a modificação pode ser feita centralizadamente na classe base.

4. Polimorfismo

Originado do grego "muitas formas", o polimorfismo é a capacidade de objetos de diferentes classes derivadas responderem à mesma mensagem (chamada de método) de maneiras distintas e adequadas à sua especificidade.

Para que o polimorfismo ocorra, é necessário haver uma interface comum (proporcionada por herança ou por contratos abstratos). O sistema pode tratar um grupo heterogêneo de objetos derivados como se fossem do tipo genérico da classe base, mas, no momento da execução, o comportamento individual de cada um será acionado.

Existem duas formas principais de manifestação conceitual:

- **Sobrescrita (Overriding)**: A subclasse reescreve a implementação de um método herdado da classe base para adaptá-lo ao seu comportamento específico. A decisão de qual método executar ocorre dinamicamente em tempo de execução (*late binding*).
- **Sobrecarga (Overloading)**: Uma mesma classe possui métodos com o mesmo nome, porém com assinaturas (parâmetros e tipos) diferentes. A escolha da execução é resolvida em tempo de compilação.

O polimorfismo reduz o acoplamento do código, pois permite escrever rotinas genéricas que funcionam com novos tipos de objetos sem a necessidade de modificar as estruturas condicionais do programa.

1.2 Documentação Sintática das Estruturas de POO

Esta seção detalha todas as palavras-chave e construções sintáticas utilizadas na implementação de POO em C++.

1.2.1 Declaração de Classe e Modificadores de Acesso

Uma classe é declarada com a palavra-chave `class`. O corpo da classe define os níveis de acesso aos seus membros:

```
1 class SerVivo {
2     protected:
3         std::string nome; // Acess vel pela classe e por classes derivadas
4                             (filhas)
5     public:
6         // Acess vel de qualquer ponto do programa
7 };
```

- `private` (Padrão): Membros são acessíveis apenas pelos métodos da própria classe.
- `protected`: Membros são inacessíveis externamente, mas acessíveis dentro da própria classe e por suas subclasses derivadas.
- `public`: Membros podem ser acessados diretamente por qualquer objeto ou função do sistema.

1.2.2 Construtores e Lista de Inicialização

O construtor possui o mesmo nome da classe e é executado automaticamente no momento da instanciação. A sintaxe de **lista de inicialização de membros** (*Member Initializer List*) atribui valores aos membros antes da execução do corpo do construtor:

```
1 // Sintaxe: Construtor(parametros) : atributo(parametro) {}
2 SerVivo(string n) : nome(n) {}
```

1.2.3 Herança Pública

A herança estabelece uma relação do tipo "É UM". Para herdar publicamente de uma classe base:

```
1 class Humano : public SerVivo {
2     public:
3         // Repassa o par metro 'n' para o construtor da classe base SerVivo
4         Humano(string n) : SerVivo(n) {}
5 };
```

1.2.4 Polimorfismo Dinâmico: virtual e override

- `virtual` (na classe base): Declara um método que pode ser sobrescrito nas classes derivadas. Habilita o *dynamic dispatch* (ligação tardia em tempo de execução).
- `override` (na classe derivada): Garante em tempo de compilação que o método está sobrescrevendo exatamente uma função virtual da classe base.

```

1 // Na Classe Base:
2 virtual void apresentar() { ... }
3
4 // Na Classe Derivada:
5 void apresentar() override { ... }

```

1.2.5 Destrutor Virtual (virtual ~Destrutor())

Sempre que uma classe possui métodos virtuais e é manipulada por meio de ponteiros da classe base, seu destrutor **deve ser virtual**. Isso garante que o destrutor correto da classe derivada seja invocado na desalocação de memória.

```

1 virtual ~SerVivo() {}

```

1.2.6 Ponteiros de Classe Base e Gerenciamento Dinâmico de Memória

O polimorfismo dinâmico exige o uso de ponteiros ou referências da classe base armazenando o endereço de instâncias das classes filhas alocadas dinamicamente via operador **new**.

```

1 // Armazenamento genérico via ponteiro da classe base
2 vector<SerVivo*> seres;
3
4 // Instancia o dinâmico
5 seres.push_back(new Humano("Arthur"));
6
7 // Chamada polimórfica via ponteiro (operador ->)
8 for (SerVivo* s : seres) {
9     s->apresentar();
10 }
11
12 // Libera a obrigatoriedade da memória alocada dinamicamente
13 for (SerVivo* s : seres) {
14     delete s;
15 }

```

1.3 Código de Referência Completo

Abaixo está a implementação de referência utilizada para exemplificação do paradigma POO em C++:

```

1 #include <iostream>
2 #include <string>
3 #include <vector>
4 using namespace std;
5
6 // CLASSE BASE (Abstrato)
7 class SerVivo {
8 protected:
9     string nome; // Encapsulamento protegido
10
11 public:
12     // Construtor com Lista de Inicialização
13     SerVivo(string n) : nome(n) {}
14
15     // Método Virtual para Polimorfismo
16     virtual void apresentar() {

```

```

17         cout << "Eu sou um ser vivo chamado " << nome << "." << endl;
18     }
19
20     // Destrutor Virtual
21     virtual ~Servivo() {}
22 };
23
24 // HERAN A: Humano herda de Servivo
25 class Humano : public Servivo {
26 public:
27     Humano(string n) : Servivo(n) {}
28
29     // SOBRESCRITA (Override)
30     void apresentar() override {
31         cout << "Ol , eu sou o humano " << nome << ", mestre das
civiliza es." << endl;
32     }
33 };
34
35 class Elfo : public Servivo {
36 public:
37     Elfo(string n) : Servivo(n) {}
38
39     void apresentar() override {
40         cout << "Sauda es , eu sou o elfo " << nome << ", guardi o
das florestas." << endl;
41     }
42 };
43
44 class Fada : public Servivo {
45 public:
46     Fada(string n) : Servivo(n) {}
47
48     void apresentar() override {
49         cout << "Encantado! Eu sou a fada " << nome << ", portadora da
magia." << endl;
50     }
51 };
52
53 int main() {
54     // Vetor de ponteiros para a classe base (Polimorfismo)
55     vector<Servivo*> seres;
56
57     // Aloca o din mica de mem ria
58     seres.push_back(new Humano("Arthur"));
59     seres.push_back(new Elfo("Legolas"));
60     seres.push_back(new Fada("Luna"));
61     seres.push_back(new Servivo("Pedro"));
62
63     cout << "=== Apresenta es no mundo de fantasia ===" << endl;
64
65     // Execu o polim rfica
66     for (Servivo* s : seres) {
67         s->apresentar();
68     }
69
70     // Desaloca o de mem ria (Good Practices)
71     for (Servivo* s : seres) {

```

```

72     delete s;
73 }
74
75     return 0;
76 }

```

Listing 1.1: Exemplo de Aplicação de POO em C++

1.4 Exercícios Práticos do Módulo

Exercício 1: Abstração e Encapsulamento (Sistema de Combate de Robôs)

Crie uma classe `Robo` contendo os atributos `modelo` (string), `versao` (int), `potenciaLaser` (float) e `integridade` (int). Implemente o método `disparar(Robo &alvo)` que reduz a integridade do alvo com base na potência do laser do atacante. Instancie dois robôs e simule um confronto.

Exercício 2: Herança e Modificadores de Acesso (Persona RPG)

Implemente a classe base `Pessoa` com atributos privados `nome` e `idade`. Crie as classes derivadas `Protagonista` (com o atributo `nivel`) e `Personagem` (com o atributo `rank`). Demonstre o acesso correto aos membros através dos construtores da classe pai.

Exercício 3: Polimorfismo Dinâmico (Membros do Inatel)

Crie a classe base `MembroInatel` com o método virtual `identificar()`. Derive as classes `Coordenador` (atributo `departamento`) e `Pesquisador` (atributo `laboratorio`). Utilize ponteiros da classe base para chamar o método `identificar()` de forma polimórfica.

Exercício 4: Polimorfismo em Coleções (Conselho das Terras Ancestrais)

Crie a classe base `MembroConselho` e as derivadas `Anao`, `Orc` e `Draconato`. Armazene instâncias dessas classes em um `std::vector<MembroConselho*>` e percorra a coleção invocando o método sobrescrito `saudar()`.

Capítulo 2

Relatório 7: Programação Orientada a Objetos em C#

2.1 Fundamentos Teóricos e Pilares da POO

No Relatório 6, os pilares da POO foram apresentados em C++, uma linguagem que exige controle manual da memória (**new** e **delete**). O C# mantém os mesmos conceitos, mas automatiza a liberação de memória por meio do **Garbage Collector** (coletor de lixo) e oferece uma sintaxe mais enxuta, principalmente no uso de **propriedades** para o encapsulamento. Os quatro pilares continuam valendo, agora ilustrados com o inventário do herói Link, do universo de *The Legend of Zelda*:

- **Abstração:** Todo objeto do jogo é modelado apenas com o que importa para o sistema. Um **Item** precisa somente de um **Nome** e de uma forma de ser usado.
- **Encapsulamento:** O acesso aos dados é controlado por propriedades. O dano de uma **Arma** pode ser lido por qualquer parte do programa, mas só pode ser alterado pela própria classe.
- **Herança:** **Arma** e **Escudo** reaproveitam a estrutura de **Item** (relação “**É UM**”: uma **Arma** é um **Item**).
- **Polimorfismo:** Uma lista do tipo **Item** guarda armas, escudos e itens comuns; ao chamar **Usar()**, cada objeto executa a sua própria versão do método.

Além dos pilares, este relatório introduz os **relacionamentos entre classes**, que descrevem como um objeto pode conter ou utilizar outros objetos (relação “**TEM UM**”):

- **Composição (relação forte):** A parte não existe sem o todo. A parte é criada **dentro** do objeto dono e “morre” junto com ele. Exemplo: o inventário de Link nasce dentro do próprio herói.
- **Agregação (relação fraca):** A parte existe de forma independente do todo. O objeto dono apenas **guarda uma referência** a um objeto criado fora dele. Exemplo: a **Master Sword** é forjada antes e só depois é adicionada ao inventário de Link.

2.2 Documentação Sintática das Estruturas de POO

Esta seção detalha todas as palavras-chave e construções sintáticas utilizadas na implementação de POO em C#. Os exercícios deste relatório devem ser resolvidos **apenas** com as estruturas documentadas aqui.

2.2.1 Estrutura do Programa e Saída no Console

Todo programa C# começa pelo método Main. As diretivas `using` importam os recursos da linguagem: `System` (para o Console) e `System.Collections.Generic` (para a `List`). O caractere `$` antes das aspas ativa a **interpolação de strings**, permitindo inserir valores entre chaves diretamente no texto.

```
1 using System;
2 using System.Collections.Generic;
3
4 public class Program
5 {
6     public static void Main(string[] args)
7     {
8         string heroi = "Link";
9         Console.WriteLine($"O herói se chama {heroi}."); // interpolação
10        Console.WriteLine("\n=== Fim ===");              // \n = quebra
11        de linha
12    }
13 }
```

2.2.2 Declaração de Classe e Propriedades

Uma classe é declarada com a palavra-chave `class`. Em C#, os atributos costumam ser expostos como **propriedades**, que controlam a leitura (`get`) e a escrita (`set`) de cada dado:

```
1 public class Magia
2 {
3     // Leitura e escrita livres, com valor inicial "Nenhum"
4     public string Efeito { get; set; } = "Nenhum";
5 }
6
7 // Dentro da classe Arma:
8 // Leitura livre, mas somente a própria classe pode alterar o valor
9 public int DanoBase { get; private set; }
10
11 // Dentro da classe Link:
12 // Campo privado: invisível fora da classe (convenção: prefixo _)
13 private List<Item> _inventario;
```

- `{ get; set; }`: A propriedade pode ser lida e alterada de qualquer ponto do programa.
- `{ get; private set; }`: A propriedade pode ser lida de qualquer ponto, mas somente os métodos da própria classe podem alterá-la. Tentar alterá-la na `Main` gera **erro de compilação**.
- `= valor`: Define o valor inicial da propriedade no momento em que o objeto é criado.
- `private`: Membro acessível apenas pelos métodos da própria classe.

2.2.3 Construtores e a Palavra-chave `this`

O construtor possui o mesmo nome da classe, não tem tipo de retorno e é executado automaticamente quando o objeto é criado com o operador `new`. A palavra-chave `this` referencia o próprio objeto que está sendo construído:

```

1 public Item(string nome)
2 {
3     this.Nome = nome;
4     Console.WriteLine($"[Item] {Nome} foi criado.");
5 }
6
7 // Instanciação: o construtor é chamado automaticamente
8 Item rupee = new Item("Rupée Azul");

```

Diferentemente do C++, não é necessário liberar a memória com `delete` nem declarar destrutores: o Garbage Collector remove automaticamente os objetos que não são mais utilizados.

2.2.4 Herança e Repasse ao Construtor Base (: base(...))

A herança é indicada por dois-pontos após o nome da classe filha. O construtor da filha repassa os parâmetros necessários ao construtor da classe pai com : `base(...)`:

```

1 public class Arma : Item // Arma É UM Item
2 {
3     public int DanoBase { get; private set; }
4
5     // "nome" vai para o construtor de Item; "dano" fica na Arma
6     public Arma(string nome, int dano) : base(nome)
7     {
8         this.DanoBase = dano;
9     }
10 }

```

2.2.5 Polimorfismo: virtual e override

- **virtual** (na classe base): Declara que o método **pode** ser reescrito pelas classes filhas. Sem ele, o C# sempre executaria a versão da classe pai.
- **override** (na classe derivada): Indica que o método está, intencionalmente, reescrevendo um método virtual da classe base.

```

1 // Na Classe Base (Item):
2 public virtual void Usar() { ... }
3
4 // Na Classe Derivada (Arma):
5 public override void Usar() { ... }

```

2.2.6 Reaproveitando o Comportamento do Pai: base.Metodo()

Dentro de um método sobrescrito, `base.Metodo()` executa a versão da classe pai. Isso permite reaproveitar parte da lógica do pai e apenas **acrescentar** o comportamento específico da filha:

```

1 public override void Usar()
2 {
3     base.Usar(); // executa primeiro o Usar() de Item
4     Console.WriteLine($"É um Escudo de Defesa: {Defesa}. Link se defende!");
5 }

```

2.2.7 Composição e Agregação

A diferença entre os dois relacionamentos está em **onde a parte é criada**:

```
1 // COMPOSIÇÃO: a Magia é criada DENTRO do construtor da Arma
2 public Arma(string nome, int dano) : base(nome)
3 {
4     this.DanoBase = dano;
5     this.MagiaAssociada = new Magia();
6 }
7
8 // AGREGAÇÃO: o Item é criado FORA de Link e apenas recebido como parâmetro
9 public void AdicionarItem(Item item)
10 {
11     this._inventario.Add(item);
12 }
```

2.2.8 Coleções Genéricas: List<T> e foreach

A List<T> é uma coleção de tamanho dinâmico, em que T é o tipo dos objetos armazenados. Uma lista do tipo da classe pai (List<Item>) aceita objetos de qualquer classe filha, o que possibilita o polimorfismo ao percorrê-la com foreach:

```
1 List<Item> inventario = new List<Item>(); // criação da lista vazia
2 inventario.Add(new Arma("Master Sword", 30)); // adiciona um objeto
3 Console.WriteLine($"Total: {inventario.Count}"); // quantidade de itens
4
5 // Cada objeto executa a SUA versão de Usar()
6 foreach (var item in inventario)
7 {
8     item.Usar();
9 }
```

- .Add(objeto): Insere um objeto no final da lista.
- .Count: Retorna a quantidade de elementos da lista.
- foreach (var x in lista): Percorre a lista elemento por elemento; a palavra var deixa o compilador deduzir o tipo da variável.
- if (a != b): Executa o bloco apenas se os valores forem diferentes.

2.3 Código de Referência Completo

Abaixo está a implementação de referência utilizada para exemplificação do paradigma POO em C#:

```
1 using System;
2 using System.Collections.Generic;
3
4 // todo: CLASSE BASE
5 // molde principal, todo item do jogo tem um nome
6 // as classes filhas vão herdar daqui
7 public class Item
8 {
9     // todo: ENCAPSULAMENTO
```

```

10 // { get; set; } é a forma do C# de expor o atributo de forma
    controlada
11 // get = pode ler, set = pode alterar
12 public string Nome { get; set; }
13
14 // todo: CONSTRUTOR
15 // roda automaticamente quando o objeto é criado
16 public Item(string nome)
17 {
18     this.Nome = nome;
19     Console.WriteLine($"[Item] {Nome} foi criado.");
20 }
21
22 // todo: VIRTUAL
23 // avisa que as classes filhas podem reescrever esse método
24 // sem isso o C# ia sempre chamar essa versão aqui, ignorando as
    filhas
25 public virtual void Usar()
26 {
27     Console.WriteLine($"\\n--- {Nome} ---");
28     Console.WriteLine("Usando um item genérico.");
29 }
30 }
31
32 // todo: CLASSE SEPARADA (sem herança)
33 // Magia não é um Item, então não herda - é só uma classe independente
34 // vai ser usada dentro de Arma mais pra frente (isso é Composição)
35 public class Magia
36 {
37     public string Efeito { get; set; } = "Nenhum";
38
39     public void Ativar()
40     {
41         Console.WriteLine($"Mágica ativada: {Efeito}!");
42     }
43 }
44
45 // todo: HERANÇA
46 // Arma herda de Item, já nasce com o atributo Nome
47 // a relação é: "Arma É UM Item"
48 public class Arma : Item
49 {
50     // private set = só a própria classe pode alterar o DanoBase
51     public int DanoBase { get; private set; }
52
53     // todo: COMPOSIÇÃO
54     // Arma tem uma Magia dentro dela
55     // quando a Arma é criada, a Magia nasce junto com ela
56     public Magia MagiaAssociada { get; set; }
57
58     // repassa o nome pro construtor do pai com : base(nome)
59     public Arma(string nome, int dano) : base(nome)
60     {
61         this.DanoBase = dano;
62         this.MagiaAssociada = new Magia();
63     }
64
65     // todo: SOBRESCRITA (override)

```

```

66 // reescreve o Usar() com o comportamento específico da Arma
67 // o override avisa o compilador que isso é intencional
68 public override void Usar()
69 {
70     Console.WriteLine($"\\n--- {Nome} ---");
71     Console.WriteLine($"É uma Arma de Dano: {DanoBase}");
72
73     if (MagiaAssociada.Efeito != "Nenhum")
74     {
75         MagiaAssociada.Ativar();
76     }
77 }
78 }
79
80 // todo: HERANÇA
81 // mesma ideia da Arma, só muda o atributo e o comportamento do Usar()
82 public class Escudo : Item
83 {
84     public int Defesa { get; private set; }
85
86     public Escudo(string nome, int defesa) : base(nome)
87     {
88         this.Defesa = defesa;
89     }
90
91     public override void Usar()
92     {
93         // todo: base.Usar()
94         // chama o Usar() do pai antes de adicionar o comportamento do
Escudo
95         // útil quando você quer reaproveitar parte da lógica da classe
pai
96         base.Usar();
97         Console.WriteLine($"É um Escudo de Defesa: {Defesa}. Link se
defende!");
98     }
99 }
100
101 public class Link
102 {
103     public string Nome { get; set; } = "Link";
104
105     // todo: COMPOSIÇÃO
106     // a lista nasce junto com Link e morre junto com ele
107     // não faz sentido existir inventário sem um herói dono dele
108     private List<Item> _inventario;
109
110     public Link()
111     {
112         this._inventario = new List<Item>();
113         Console.WriteLine($"\\n[Link] O herói {Nome} foi criado.");
114     }
115
116     // todo: AGREGAÇÃO
117     // os itens existem fora de Link, ele só guarda uma referência pra
eles
118     // masterSword e hyliaShield foram criados antes e vivem
independente do herói

```

```

119     public void AdicionarItem(Item item)
120     {
121         this._inventario.Add(item);
122     }
123
124     public void MostrarInventario()
125     {
126         Console.WriteLine($"\\n{Nome} tem {_inventario.Count} itens no
127         seu inventário:");
128
129         // todo: POLIMORFISMO EM AÇÃO
130         // o foreach trata tudo como Item, mas cada objeto roda o seu pr
131         óprio Usar()
132         foreach (var item in _inventario)
133         {
134             item.Usar();
135         }
136     }
137 }
138
139 public class Program
140 {
141     public static void Main(string[] args)
142     {
143         Console.WriteLine("=== Iniciando Aventura em Hyrule ===");
144
145         Arma masterSword = new Arma("Master Sword", 30);
146         masterSword.MagiaAssociada.Efeito = "Corte de Energia Sagrada";
147
148         Escudo hylianShield = new Escudo("Hylian Shield", 50);
149
150         Link linkHeroi = new Link();
151
152         linkHeroi.AdicionarItem(masterSword);
153         linkHeroi.AdicionarItem(hylianShield);
154         linkHeroi.AdicionarItem(new Item("Rupree Azul"));
155
156         linkHeroi.MostrarInventario();
157
158         Console.WriteLine("\\n=== Fim da Demonstração ===");
159     }
160 }

```

Listing 2.1: Exemplo de Aplicação de POO em C#

2.4 Exercícios Práticos do Módulo

Exercício 1: Encapsulamento e Construtores (Defesa de Minas Tirith)

Durante o cerco a Minas Tirith, cada combatente convocado deve ser registrado com seu nome, povo, posto e o número do Círculo da cidade (de 1 a 7) que irá defender. Crie a classe `CombatenteDeGondor`:

- Propriedades `Nome`, `Povo`, `Posto` (string) e `Circulo` (int), todas com `private set`, inicializadas por um construtor que imprime uma mensagem de convocação.
- Propriedade `Armamento` com `private set` e valor inicial “Desarmado”, alterada somente pelo método `Equipar(string arma)`.
- Método `ApresentarUnidade()`, que exibe os dados do combatente e só exibe o armamento quando ele for diferente de “Desarmado”.

Na `Main`, instancie Legolas (Elfo, Arqueiro, Círculo 1) equipado com o “Arco dos Galadhrim”, Peregrin Took (Hobbit, Guarda da Cidadela, Círculo 7) sem armamento e um terceiro combatente à escolha. Chame `ApresentarUnidade()` em todos. Por fim, tente alterar `Posto` diretamente na `Main`, observe o erro de compilação e comente a linha.

Exercício 2: Herança e Polimorfismo (Batalha de Exibição Pokémon)

Crie a classe base `Pokemon` com as propriedades `Especie` (string) e `Nivel` (int, com `private set`), ambas recebidas no construtor, e o método virtual `EntrarEmCampo()`, que imprime a espécie, o nível e um ataque genérico (Investida). Derive as classes:

- `TipoPlanta`, com a propriedade `GolpeEspecial` (string). Sobrescreva `EntrarEmCampo()` sem chamar a versão do pai.
- `TipoEletrico`, com a propriedade `Voltagem` (int, com `private set`). Sobrescreva `EntrarEmCampo()` chamando `base.EntrarEmCampo()` antes de exibir a descarga elétrica.

Na `Main`, crie uma `List<Pokemon>` com um `Sceptile` (`TipoPlanta`), um `Jolteon` (`TipoEletrico`) e um `Eevee` (`Pokemon` genérico). Exiba a quantidade de Pokémon em campo com `Count` e percorra a lista com `foreach` chamando `EntrarEmCampo()`.

Exercício 3: Composição e Agregação (O Grimório de Frieren)

A elfa Frieren carrega um grimório que nasce com ela e registra os feitiços que ela aprende (**composição**), e viaja com companheiros que existem independentemente dela (**agregação**). Implemente:

- **Feitico**: propriedade **Nome** e método **Conjurar()**.
- **Grimorio**: lista privada de **Feitico** criada no construtor; método **Registrar(string nomeFeitico)**, que cria o **Feitico** **dentro** do grimório; e método **ListarFeiticos()**, que exibe a quantidade e conjura cada um.
- **Companheiro**: propriedades **Nome** e **Funcao**, e método **Apresentar()**.
- **Maga**: propriedade **Nome**, propriedade **Grimorio** (com **private set**) criada no construtor, lista privada de **Companheiro** e os métodos **RecrutarCompanheiro(Companheiro c)** e **MostrarGrupo()**.

Na **Main**, crie **Fern** (**Maga Aprendiz**) e **Stark** (**Guerreiro**) **antes** de **Frieren**, recrute ambos, registre três feitiços pelo grimório de **Frieren**, exiba o grupo e o grimório e, por fim, chame **Apresentar()** diretamente em **Stark**. Explique, em um comentário no código, qual relacionamento é composição e qual é agregação.

Exercício 4: Revisão Geral (Arquivos Proibidos da Miskatonic)

O bibliotecário **Henry Armitage** cataloga os relatos de entidades cósmicas guardados na **Universidade Miskatonic**. Implemente:

- **EntidadeCosmica** (classe base, **não** abstrata): propriedades **Nome** e **Origem** (valor inicial “Desconhecida”), construtor que imprime o registro da entidade e método virtual **Manifestar()**, que só exibe a origem quando ela for diferente de “Desconhecida”.
- **Profundo**: propriedade **Profundidade** (**int**, com **private set**); sobreescreve **Manifestar()** **sem** chamar o pai.
- **MiGo**: propriedade **Artefato** (**string**); sobreescreve **Manifestar()** chamando **base.Manifestar()** antes do seu comportamento.
- **Pesquisador**: propriedade **Nome**, lista privada de **EntidadeCosmica** criada no construtor (composição), método **Catalogar(EntidadeCosmica e)** (agregação) e método **LerCatalogo()** (polimorfismo com **foreach**).

Na **Main**, crie um **Profundo**, um **MiGo** com **Origem** igual a “Yuggoth” e uma **EntidadeCosmica** genérica (“A Cor que Caiu do Espaço”); catalogue as três no pesquisador **Henry Armitage** e chame **LerCatalogo()**.